



Hook CMS — Knowledge Transfer

A theme-driven Content Management System that controls the design, layout, components and content of a separate PWA — without ever touching the PWA's source code. This document is the technical handover: architecture, data model, API surface, how to operate it, and how to extend it.

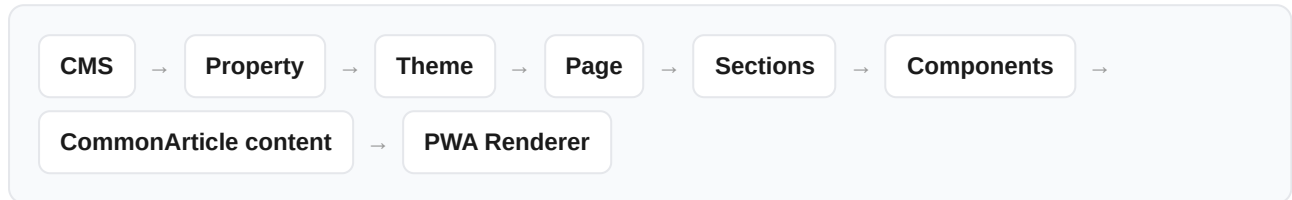
cms-pwa-platform · FastAPI + MongoDB + Next.js (CMS) + Next.js (PWA)
Prepared for engineering handover / onboarding

Contents

- | | |
|------------------------------------|---|
| 1 System Overview | 8 Backend API Reference |
| 2 Technology Stack | 9 How the PWA Renders CMS Config |
| 3 Repository Structure | 10 Auth & Roles |
| 4 Data Model | 11 CMS Screens Guide |
| 5 Theme System | 12 Running & Deploying |
| 6 Component System | 13 Extending the System |
| 7 Page Builder & Publishing | 14 Known Simplifications |
| | 15 Demo Script |

1. System Overview

The platform separates **five concerns** that are normally tangled together in a typical CMS: content, page structure, components, component variants, and theme/design tokens. A single change in the CMS — activate a different theme, publish a new page layout, edit an article — is reflected in the PWA on the next request, with zero PWA code changes and zero redeploy.



The core rule

The CMS controls **WHAT** (content), **WHERE** (page/sections), **WHICH COMPONENT**, **WHICH VARIANT**, and **WHICH THEME**. The PWA controls only **HOW a component actually renders**. A theme is design tokens and a variant mapping — never markup, never code. A page section is `{ type, variant, data_source, config, props }` — never HTML.

Three applications, one contract

Backend (FastAPI)

Owns MongoDB, exposes REST APIs, resolves data sources into `CommonArticle` documents, and assembles two purpose-built endpoints the PWA depends on:
`/api/pwa/config/:propertySlug` and
`/api/pwa/page/:propertySlug/:pageSlug`.

CMS (Next.js, port 3000)

Admin app: theme editor with live preview, drag-and-drop Page Builder, article management, navigation editor, properties, version history. Everything here writes to the backend.

PWA (Next.js, port 3001)

Public-facing reader app. Every route is a thin server component that fetches the two `/api/pwa/*` endpoints and renders through a component registry — it never imports anything CMS-specific.

Shared packages

`packages/shared-types` (TypeScript types used by both frontends) and
`packages/component-schema` (static palette metadata for the Page Builder).

2. Technology Stack

Layer	Stack
CMS frontend	Next.js 16 (App Router, Turbopack), TypeScript, Tailwind CSS v4, shadcn/ui (Radix base), React Hook Form + Zod, dnd-kit, lucide-react, sonner
PWA frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS v4, server components only (no client state needed for rendering)
Backend	Python 3.11, FastAPI, Pydantic v2, Motor (async MongoDB driver), python-jose + passlib (JWT/bcrypt), redis.asyncio
Database	MongoDB (collections listed in §4)
Cache	Redis — short-TTL (15s) cache on the two <code>/api/pwa/*</code> endpoints only, invalidated on publish/activate
Infra	npm workspaces monorepo, Docker + docker-compose, per-app Dockerfiles

3. Repository Structure

4. Data Model

All backend models live in `backend/app/models/*.py` as Pydantic classes. MongoDB collections:

Collection	Purpose	Key fields
users	CMS accounts	email, password_hash, role
properties	Brand/edition (Hook, Hook Hindi, Hook English)	slug, active_theme_id, active_home_page_id, status
themes	Design tokens + variant mapping	design_tokens{colors, typography, spacing, radius, shadows, breakpoints}, component_mapping, status
components	Component type registry	type, variants[], fields[] (dynamic form schema), status
pages	Page structure	sections[] (draft), published_sections[] (live), status, published_at, seo
navigations	Top / bottom / sidebar nav	property_id, type, items[]
common_articles	Content — ported 1:1 from the pre-existing Mongoose CommonArticleSchema	script_headline, script_slug, script_content, parent_category, tags, is_trending, is_breaking, publication_date, ...(~90 fields, untouched by the design system)
versions	Generic snapshot log	entity_type (theme/page/navigation), entity_id, version, snapshot, created_by, created_at

Section shape

A page's `sections[]` array is the entire layout description. Never HTML, always this shape:

```
{
  "id": "s1",
  "type": "news_grid",
  "variant": "three_column",
  "title": "Latest News",
  "data_source": { "type": "latest", "limit": 10 },
  "config": {
    "columns": { "mobile": 1, "tablet": 2, "desktop": 3 },
    "spacing": { "top": 20, "bottom": 20 }
  },
  "props": { "showExcerpt": false }
}
```

Data source types

latest trending breaking category tag manual video search

Resolved server-side in `backend/app/repositories/article_repo.py::resolve_data_source` — every section that declares a `data_source` arrives at the PWA with its `items[]` already populated; the PWA never queries articles directly for page sections.

5. Theme System

A theme is **only** design tokens plus a component-variant mapping:

```
{
  "design_tokens": {
    "colors": { "primary": "#E11D48", "secondary": "#0F172A", "background": "#FFFFFF",
      "surface": "#F8FAFC", "text": "#111827", "muted": "#64748B" },
    "typography": { "fontFamily": "Inter", "headingWeight": 700, "bodyWeight": 400 },
    "spacing": { "xs": 4, "sm": 8, "md": 16, "lg": 24 },
    "radius": { "small": 6, "medium": 12, "large": 20 },
    "shadows": { "sm": "...", "md": "...", "lg": "..." }
  },
  "component_mapping": { "hero": "split", "news_card": "vertical", "carousel": "cards" }
}
```

Editing flow (CMS)

`apps/cms/components/themes/theme-editor-client.tsx` renders tabs — General, Colors, Typography, Spacing, Radius, Shadows, Components, Navigation — each editing a slice of the same in-memory token object. A live preview (`theme-preview.tsx`) re-renders a mock Header/Hero/Cards/Carousel/Footer instantly from that same state, before Save.

Applying a theme (PWA)

`apps/pwa/renderer/theme-provider.tsx` converts the active theme's tokens into CSS custom properties (`--color-primary` , `--radius-medium` , ...) injected via a `<style>` tag at the top of every page. Every PWA component reads exclusively from these variables — never a hardcoded color or radius.

Activation

`POST /api/themes/:id/activate?property_id=...` sets `status=active` on the theme and `active_theme_id` on the property, snapshots a version, and clears the PWA cache. The PWA reflects it on the very next request — no deploy.

6. Component System

15 component types are seeded, each with variants and a dynamic field schema:

```
hero, banner, news_card, news_list, news_grid, carousel, video, image,
text, category, ad, spacer, header, footer, bottom_navigation
```

Field schema (dynamic forms)

Each component document carries a `fields[]` array the Page Builder's properties panel renders *dynamically* — this is what lets you add a new field to a component without touching any builder code:

```
{
  "name": "buttonText", "label": "Button Text", "type": "text", "required": false
}
```

Supported field types: `text` `textarea` `number` `boolean` `select` `multiselect` `image` `video` `url`
`color` `article` `category` `tag` `date` `datetime`

Where each half lives

Concern	Location
Field/variant schema (source of truth)	MongoDB <code>components</code> collection
Palette icon/label/grouping (CMS-only presentation)	<code>packages/component-schema/src/index.ts</code>
Dynamic property-panel rendering	<code>apps/cms/components/page-builder/properties-panel.tsx</code>
Actual React rendering in the PWA	<code>apps/pwa/components/*.tsx</code> , registered in <code>apps/pwa/renderer/component-registry.tsx</code>

7. Page Builder & Publishing

`apps/cms/components/page-builder/page-builder-client.tsx` — a 3-pane `dnd-kit` layout:

- **Palette** (left) — draggable *and* click-to-add component cards, grouped Layout / Content / Navigation.
- **Canvas** (center) — a sortable list of section cards (drag handle, delete, duplicate); dropping a palette item anywhere on the canvas appends a new section.
- **Properties panel** (right) — for the selected section: title, variant, dynamic content fields (from the component's `fields[]`), data source config, responsive column counts, top/bottom spacing.

Draft vs. Published

Every page document stores **two** section arrays: `sections` (the working draft) and `published_sections` (what the PWA actually serves). `PUT /api/pages/:id` only ever touches `sections`. `POST /api/pages/:id/publish` copies `sections` → `published_sections`, stamps `published_at`, and snapshots a version. The PWA's `/api/pwa/page/...` endpoint reads *only* `published_sections` — so draft edits never leak, by construction.

Preview

`GET /api/pages/:id/preview` resolves the current *draft* sections (with real article data) for the builder's Preview dialog, without publishing anything.

Version history

Every save/publish/activate snapshots the **pre-change** state into the generic `versions` collection (entity_type + entity_id + incrementing version number). The Version History dialog (shared by Theme editor and Page Builder) lists and restores any prior snapshot via `POST /api/versions/:type/:id/:version/restore`.

8. Backend API Reference

All routes prefixed `/api`. JWT bearer auth required except `/api/auth/login` and everything under `/api/pwa/*`.

Method & Path	Notes
<code>POST /auth/login</code>	Returns JWT + user
<code>GET /auth/me</code>	Current user from token
<code>GET/POST /properties</code> , <code>GET/PUT /properties/:id</code>	Admin only for writes
<code>GET/POST /themes</code> , <code>GET/PUT /themes/:id</code>	designer role for writes
<code>POST /themes/:id/activate?property_id=</code>	Sets active theme for a property
<code>GET /themes/:id/versions</code>	
<code>GET/POST /components</code> , <code>GET/PUT /components/:id</code>	designer role for writes
<code>GET/POST /pages (?property_id=)</code> , <code>GET/PUT/DELETE /pages/:id</code>	editor role for writes
<code>POST /pages/:id/publish</code> , <code>POST /pages/:id/duplicate</code> , <code>GET /pages/:id/preview</code>	
<code>GET /pages/:id/versions</code>	
<code>GET /navigation?property_id=</code> , <code>POST /navigation</code> , <code>PUT /navigation/:id</code>	designer/editor roles for writes
<code>GET /articles</code> , <code>GET /articles/trending latest category/:slug slug/:slug</code>	
<code>GET/POST /articles</code> , <code>GET/PUT /articles/:id</code>	editor role for writes
<code>GET /versions/:entityType/:entityId</code>	theme / page / navigation
<code>POST /versions/:entityType/:entityId/:version/restore</code>	editor/designer role
<code>GET /pwa/config/:propertySlug</code>	Public. property + active theme tokens + navigation
<code>GET /pwa/page/:propertySlug/:pageSlug</code>	Public. page meta + sections, each pre-resolved with component def + article items

9. How the PWA Renders CMS Config

Fetch `/api/pwa/config/:propertySlug`

Fetch `/api/pwa/page/:propertySlug/:pageSlug`

ThemeProvider — tokens → CSS variables

PageRenderer

SectionRenderer (per section)

`componentRegistry[section.type]`

Rendered component, styled only from CSS vars

Every PWA route (`apps/pwa/renderer/page-route.tsx`) is a thin server component using `fetch(..., { cache: "no-store" })` — confirmed in production builds, every route is marked dynamic (*f*), so nothing is frozen at build time. The PWA imports nothing from the CMS's database structure; it only ever calls the two `/api/pwa/*` endpoints.

Routes

```
/                → page slug "home"
/trending, /latest-news, /sports, /entertainment, /videos, /search
/article/[slug]  → fetches the article directly + the "article-detail" template's
                  header/footer/related sections
```

10. Auth & Roles

Single `users` collection, JWT bearer tokens (HS256, configurable expiry). `backend/app/core/deps.py` exposes `get_current_user` and `require_role(*roles)`.

Role	Can write
admin	everything
editor	articles, pages
designer	themes, components
viewer	read-only

Demo login: `shailendra@hook.online` / `admin123` (seeded by `backend/app/scripts/seed.py`).

11. CMS Screens Guide

Screen	Route	What it does
Dashboard	<code>/dashboard</code>	Article/page/theme counts, published vs draft, trending count, recent articles
Articles	<code>/articles</code> , <code>/articles/new</code> , <code>/articles/:id/edit</code>	Tabbed form (General/Media/Categories/Video/SEO/Social/Publishing/Advanced) over a curated subset of <code>CommonArticle</code> fields
Themes	<code>/themes</code> , <code>/themes/:id</code>	List + tabbed editor with live preview and Activate action
Components	<code>/components</code>	List/detail view of types, variants, fields; active/inactive toggle
Pages	<code>/pages</code> , <code>/pages/:id/builder</code>	List with publish/duplicate/delete; the drag-and-drop Page Builder
Navigation	<code>/navigation</code>	Top/bottom/sidebar nav editor with drag-to-reorder
Properties	<code>/properties</code>	Property CRUD, active theme / active home page selection

12. Running & Deploying

Local development

```
# Backend (port 8010)
cd backend && source .venv/bin/activate
uvicorn app.main:app --reload --port 8010

# Seed demo data (run once, or anytime to reset)
python -m app.scripts.seed

# CMS (port 3000)
npm run dev:cms

# PWA (port 3001)
npm run dev:pwa
```

Docker

```
docker compose up --build
docker compose exec backend python -m app.scripts.seed
```

Services: `mongo`, `redis`, `backend`, `cms`, `pwa` — see `docker-compose.yml`.

Environment variables

Copy `.env.example` to `.env` (root) and `backend/.env`. Each frontend also has its own `.env.local` pointing at the backend (default `http://localhost:8010/api`).

13. Extending the System

Add a new component type

1. Insert a document into `components` (via `POST /api/components` or the seed script) with a unique `type`, its `variants`, and its `fields`.
2. Add a palette entry in `packages/component-schema/src/index.ts` (icon + label + group).
3. Implement a React component in `apps/pwa/components/` reading `section.props`, `section.variant`, `section.items` — styled only from CSS variables.
4. Register it in `apps/pwa/renderer/component-registry.tsx` under the same `type` key.

No other screen or route needs to change — the property panel and the PWA renderer are both fully data-driven from steps 1 and 4.

Add a new theme

`POST /api/themes` (or "+ Create Theme" in the CMS) → fill in `design_tokens` and `component_mapping` in the editor → activate for a property. The PWA picks it up on the next request.

Add a new page

Create via the Pages screen or `POST /api/pages`, build it in the Page Builder, Publish. Add a matching route folder under `apps/pwa/app/` if it needs its own URL (a one-line wrapper around `PageRoute`).

14. Known Simplifications

Deliberate MVP-scope trade-offs — flag before assuming production-readiness:

- **No media upload.** Image/video fields are plain URL strings; no asset manager.
- **No component field-schema builder UI.** Field schemas are authoritative in MongoDB (seeded); the CMS only *consumes* them in the Page Builder, it doesn't let you visually design new fields.
- **Components screen is view/toggle-only** — not a full CRUD schema editor.
- **Search page is static** — a CMS-configured section, not a live query against the search box.
- **RBAC is enforced but lightly tested** — only an admin user is seeded; non-admin role paths are implemented but not exercised end-to-end.
- **No automated test suite** — verification was via direct API smoke tests, ``tsc --noEmit``, and production ``next build`` for both frontends.
- **Article update semantics:** the generic repository update strips `None` values before `$set`, so a field can't currently be explicitly cleared to null via the API — only to an empty string.

15. Demo Script

The single most important thing to show a stakeholder:

1. Log into the CMS (`shailendra@hook.online` / `admin123`).
2. Open **Themes** → **Hook Modern**, change the primary color and the `news_card` variant, **Save**.
3. Open the PWA and refresh — color and card layout changed, zero PWA code touched.
4. Back in the CMS, open **Pages** → **Home** → **Edit**, drag or click in a new section, configure its data source, **Publish**.
5. Refresh the PWA — the new section appears.
6. In **Themes**, click **Activate** on **Hook Classic** for the Hook property. Refresh the PWA — same articles, completely different look (navy/gold serif vs. rose/slate sans) — because only the active-theme pointer changed.

This is the proof of the whole architecture: **one CMS** → **multiple themes** → **multiple page layouts** → **the same content** → **multiple PWA designs, with no PWA source-code change**.